

Transaction Processing

Scenario 1: ATM Transaction Processing

A customer visits an ATM to withdraw 10,000Rs from their savings account (**Account A**). The account currently holds a balance of 15,000Rs. The withdrawal process involves a series of steps managed by the bank's transaction processing system.

However, there is a possibility that the ATM might fail to dispense the cash due to a mechanical issue **after** the amount has been deducted from the account. The system should ensure that the customer is not charged if the cash is not dispensed, and the database remains consistent.

Task:

As a database administrator, explain how this transaction can be handled using SQL transaction control commands while ensuring all ACID properties (Atomicity, Consistency, Isolation, Durability) are maintained.

Solution:

Time	Transaction	Balance of Account A
t_1	BEGIN TRANSACTION	15,000
t_2	↓ read balance	15,000
t_3	↓ balance = balance - 10000	5,000
t_4	↓ SAVEPOINT after_debit	5,000
t_5	↓ try to dispense cash	5,000
t_6	↓ ROLLBACK TO after_debit	5,000
t_7	↓ INSERT INTO ATM_ErrorLog VALUES ('Dispense Failure');	5,000
t_8	↓ COMMIT will done since last <i>savepoint</i> will be saved.)	5,000

The following table is provided for your understanding; you are only required to write a solution in the same way as demonstrated above.

Time	Transaction Step	SQL Command
t ₁	Begin ATM transaction	BEGIN TRANSACTION;
t ₂	Check account balance of Account A (15,000Rs)	SELECT balance FROM accounts WHERE acc_no = 'A';
t ₃	Set a savepoint before debiting	SAVEPOINT before_debit;
t ₄	Deduct 10,000Rs from Account A	UPDATE accounts SET balance = balance - 10000 WHERE acc_no = 'A';
t ₅	ATM attempts to dispense cash	(System process - no SQL)
t ₆ (fail)	ATM fails to dispense cash	ROLLBACK TO SAVEPOINT before_debit;
t ₇ (fail)	End transaction	COMMIT;
t ₆ (pass)	ATM successfully dispenses cash	(System process - no SQL)
t ₇ (pass)	Commit the transaction	COMMIT;

Scenario 2 : Online Shopping Cart - Payment Failure

A customer places an order for items worth Rs. 8,000. Before confirming the order, the system reduces item stock and records the order in the database. However, the payment gateway fails. The system should roll back all actions, including stock deduction and order record.

Solution:

Time	Transaction	Order Status / Inventory
t ₁	BEGIN TRANSACTION	item stock = 10
t ₂	UPDATE Inventory SET stock = stock - 1	item stock = 9
t ₃	INSERT INTO Orders (user_id, item_id, amount) VALUES (...)	Order placed (pending)
t ₄	SAVEPOINT before_payment	item stock = 9
t ₅	Try to process payment	stock = 9
t ₆	Payment fails → ROLLBACK TO before_payment	item stock = 9
t ₇	INSERT INTO OrderErrorLog VALUES ('Payment Failed')	Log added
t ₈	COMMIT	Order not placed, stock safe

Scenario 3: Fee Submission - Network Failure During Notification

A student pays Rs. 20,000 in fees. The amount is deducted and recorded, but SMS/email notification fails. The system must not reverse the transaction but should log the notification failure.

Solution:

Time	Transaction	Account Balance / Logs
t ₁	BEGIN TRANSACTION	Account = 25,000
t ₂	UPDATE Accounts SET balance = balance - 20,000	Account = 5,000
t ₃	INSERT INTO FeePayments (student_id, amount) VALUES (...)	Payment recorded
t ₄	SAVEPOINT before_notify	Account = 5,000
t ₅	Try to send SMS/Email notification	Account = 5,000
t ₆	Notification fails → ROLLBACK TO before_notify	Payment remains
t ₇	INSERT INTO NotificationErrors VALUES ('SMS Failed')	Error logged
t ₈	COMMIT	Final balance = 5,000

Scenario 4: Electricity Bill Payment – System Crash Recovery

A customer pays their electricity bill online (Rs. 7,500). The transaction deducts money from the account and updates the bill status to “Paid.” However, the system crashes **after the deduction but before the status update is committed**.

During recovery, the system must decide whether to **undo or redo** the transaction based on the transaction log.

Solution:

Time	Transaction	Bill Status / Balance
t ₁	BEGIN TRANSACTION	Balance = 10,000
t ₂	Deduct Rs. 7,500 from user account	Balance = 2,500
t ₃	Log write operation → UPDATE Accounts SET balance = 2,500	Log entry created
t ₄	Update bill status to "Paid"	Status = Paid
t ₅	System Crash before COMMIT	Recovery needed
t ₆	Recovery Process Starts	
t ₇	Check logs: Transaction not committed	
t ₈	Perform UNDO (rollback deduction)	Balance = 10,000
t ₉	Bill status remains "Unpaid"	Status = Unpaid

How Recovery Works:

- **UNDO:** Because the transaction wasn't committed, any changes are **rolled back** using the **log records**.
- If it had been committed before crash, a **REDO** would be applied.

ACID Properties Ensured:

- **Atomicity:** Either both deduction and status update happen or none.
- **Consistency:** No inconsistent billing status (e.g., money deducted but not marked paid).
- **Durability:** Committed changes would survive crash (via REDO), but uncommitted are undone.